



전자전기컴퓨터설계실험II

실험 전 보고서

조합회로 실습

실험 01-10

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 09월 13일
대상	논리 게이트 · 전가산기 · 4비트 가산기 · 4비트 감산기 · 4비트 크 기 비교기 · 4:1 멀티플렉서 · 1:8 디멀티플렉서 · 8:3 인코더 · 3:8 디코더 · 7세그먼트 디코더
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab1>

제출 태그: [lab1-pre-v1](#)

커밋 해시: [af793e4dc372d3928a67eac36062a973887187ba](#)

```

tag lab1-pre-v1
tagger: a40893632-a1 <a40893632@gmail.com>
taggerDate: Mon Sep 14 17:31:03 2026 +0900
LAB1 pre-Lab submission with XDC
commit af793e4dc372d3928a67aac36862a973897187ba (HEAD -> main, tag: lab1-pre-v1, origin/main)
Author: a40893632-a1 <a40893632@gmail.com>
AuthorDate: Mon Sep 14 17:27:58 2026 +0900
Commit: a40893632-a1 <a40893632@gmail.com>
CommitDate: Mon Sep 14 17:27:58 2026 +0900

```

그림 0: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

Lab1 실험의 목적은 Verilog HDL을 이용하여 디지털 논리 설계 및 마이크로프로세서에서 배운 기본적인 조합논리회로를 설계하고, 테스트 벤치를 이용해 모든 입력 조합에 대한 출력 값을 검증하는 것이다. 이때 Icarus Verilog로 시뮬레이션하고 VaporView에서 파형을 확인하는 과정을 거친다. 또한 예상한 진리표와 실제 시뮬레이션 결과를 비교하여 각 회로의 동작 원리를 이해하고, 실제 FPGA 보드 검증을 통해 실험의 완성도를 높인다.

실험 범위

	회로	목적
1	논리 게이트	AND, OR, XOR 연산
2	전가산기	반가산기 2개로 세 입력의 합과 올림 구현
3	4비트 가산기	두 4비트 수의 합과 자리올림 확인
4	4비트 감산기	두 4비트 수의 차와 빌림 확인
5	4비트 비교기	두 수의 큼 · 같음 · 작음 판별
6	4:1 멀티플렉서	선택 신호에 따른 입력 선택
7	1:8 디멀티플렉서	하나의 입력을 선택된 출력으로 전달
8	8:3 인코더	one-hot 입력을 3비트 번호로 변환
9	3:8 디코더	3비트 입력을 One-hot 출력으로 변환
10	7세그먼트 디코더	4비트 입력을 표시용 점등 패턴으로 변환

2. 관련 이론

조합논리회로

조합논리회로란 현재 입력되는 값에 의해서 출력이 결정되는 논리회로로, 순차회로와 달리 기억장치가 없다. 기본 논리 게이트인 AND, OR, NOT의 조합으로 이뤄지고, Lab1에서 구현하는 가산기, 감산기, 비교기 등 모두 조합논리회로에 해당한다.

FPGA와 ASIC의 차이

FPGA는 제조 이후에도 비트스트림을 이용하여 내부 논리 기능과 연결 구조를 설정할 수 있는 집적회로이다. 반면 ASIC은 특정 용도에 맞게 물리적인 회로를 설계하여 제조하므로 제작 이후에 내부 구조를 쉽게 변경할 수 없다. FPGA는 설계 변경과 반복 검증에 유리하지만, 논리 블록과 배선 구조가 추가로 필요하다. 동일한 기능과 공정 기준으로 비교하면 ASIC은 FPGA보다 일반적으로 면적, 속도 및 전력 효율에서 유리하다[1]. 반면 FPGA는 제조 후에도 회로 기능을 다시 설정할 수 있다는 큰 이점이 있다[2].

반도체 공정과 관련된 FPGA 사용 이유

1. 별도의 반도체 제작 없이 설계를 변경할 수 있다.

-FPGA는 이미 제조된 범용 칩에 비트스트림을 입력하여 회로를 구현하므로 설계를 변경할 때마다 새로운 집적회로를 제작할 필요가 없다. 반면, ASIC은 설계 내용이 반도체 구조에 반영되므로 제조 이후에 설계를 바꾸려면 새로운 제작 과정이 필요할 수 있다. 따라서 FPGA는 설계 변경이 많거나 반복 검증이 필요한 시제품 제작에 적합하다[2].

2. 제조 이후 발견된 논리 오류를 수정할 수 있다.

-FPGA는 논리 오류나 사양 변경 등의 이슈가 발생하더라도 설계 파일을 수정한 뒤 새로운 비트스트림을 다운로드 하여 회로를 변경할 수 있다. 이러한 변경은 FPGA가 제공하는 논리 및 배선 자원의 범위 안에서 가능하므로 반도체를 다시 제작하지 않고 설계를 수정할 수 있어 재제작 위험을 줄이는데 유리하다[2].

3. 개발 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
코드 작성	VS Code 1.137.0
HDL Simulator	Icarus Verilog 13.0
파형 확인	VaporView
버전 관리	Git 2.55.0. windows.5
실행 Script	Python 3.14.7
FPGA 도구	AMD Vivado 2026.1

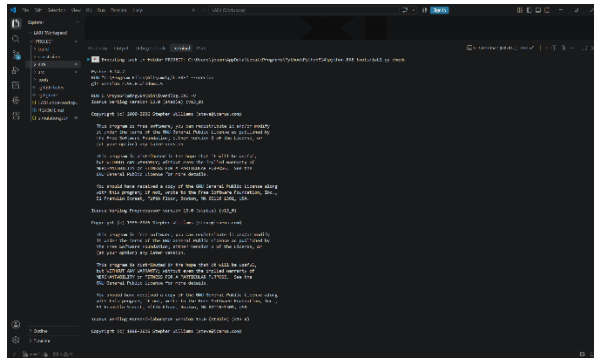


그림 1: VS Code 워크스페이스 및 도구 버전

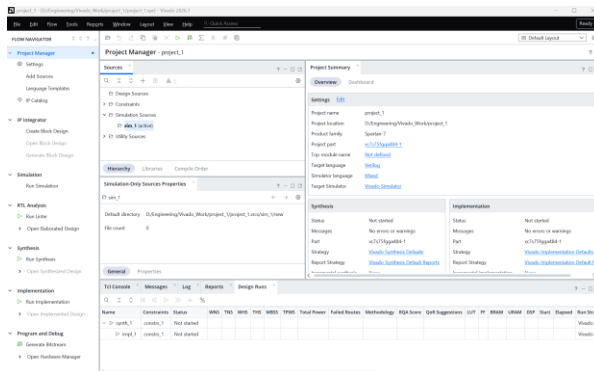


그림 2: Vivado 2026.1 개발 환경

파일의 역할

src 폴더에는 FPGA에 구현할 RTL 설계 파일인 .v파일을 작성하였다. Sim 폴더에는 회로에 입력을 인가하고 예상 출력과 실제 출력을 자동으로 비교하는 테스트벤치 .sv 파일을 작성하였다. Simulation.json에는 시뮬레이션에서 사용할 RTL 파일, 테스트벤치 파일 및 시뮬레이션 top 모듈을 지정하였다.

공통 시뮬레이션 방법

File → Save All / Terminal → Run Task / 02 Simulate / PASS 확인 및 wave.vcd 생성 / 03 Open waveform / VaporView 파형 분석 과정을 통해 시뮬레이션을 진행하였다.

4. 실험 1 · 논리 게이트

핵심 내용

·설계 파일: logic_gate.v

·테스트벤치: tb_logic_gate_modern.sv

·입력: a, b / ·출력: x(AND), y(OR), z(XOR)

·검증 조건: 4가지 입력 조합, 10ns 간격, 총 40ns

핵심 코드

```

9      assign x = a & b;
10     assign y = a | b;
11     assign z = a ^ b;
  
```

그림 3: 논리 게이트 Verilog 핵심 코드

예상 결과

	a	b	x(AND)	y(OR)	z(XOR)
0	0	0	0	0	0
0	0	1	0	1	1
1	0	0	0	1	1
1	1	1	1	1	0

&는 두 입력이 모두 1일 때만 1을 출력하고, | 는 하나 이상의 입력이 1일 때 1을 출력한다. ^는 두 입력이 서로 다를 때, 즉 하나만 1인 경우 1을 출력한다.

결과 설명

LAB1_PASS logic_gate cases=4
D:\Engineering\Verilog_Lab\LAB1\lab1_01_logic_gates\sim\tb_logic_gate_modern.sv:53: \$finish called at 40000 (ps)

그림 4: 논리 게이트 시뮬레이션 실행 결과(PASS)

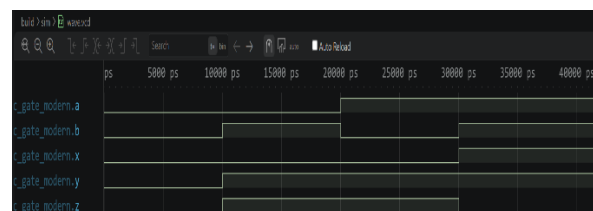


그림 5: 논리 게이트 입출력 파형

입력이 00-01-10-11 순으로 10ns 간격으로 인가되었다. AND 출력은 두 입력이 모두 1일 때만, OR 출력은 하나 이상의 입력이 1일 때, XOR 출력은 두 입력이 서로 다른 01,10에서만 각각 1이 되었고, 예상 진리표와 일치하였다.

5. 실험 2 · 전가산기

핵심 내용

·설계 파일: half_adder.v , full_adder.v

·테스트벤치: tb_full_adder.sv

·입력: a, b, Cin / ·출력: s, Cout

·검증 조건: 8가지 입력 조합, 10ns 간격, 총 80ns

핵심 코드

```

15 half_adder first (.a(a), .b(b), .s(partial_sum), .c(carry_ab));
16 half_adder second (.a(partial_sum), .b(cin), .s(s), .c(carry_cin));
17
18 assign cout = carry_ab | carry_cin;

```

그림 6: 전가산기 Verilog 핵심 코드

예상 결과

a	b	Cin	Cout	s
0	0	0	0	0
0	1	1	1	0
1	1	1	1	1

첫 번째 반가산기는 a와 b의 덧셈, 두 번째 반가산기는 첫 번째 합 s(partial_sum)과 입력 자리올림 Cin을 더한다. 두 반가산기에서 발생한 자리올림을 OR 연산하여 최종 Cout을 만든다.

결과 설명

```

LAB1_PASS full_adder cases=8
D:\Engineering\Verilog_Lab\LAB1\lab1_02_full_adder\sim\tb_full_adder.sv:57: $finish called at 80000 (1ps)

```

그림 7: 전가산기 시뮬레이션 실행 결과(PASS)

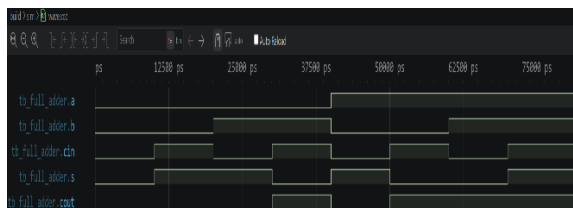


그림 8: 전가산기의 세 입력과 합, 자리올림 출력 파형

30~40ns에서 a=0, b=1, Cin=1 이므로 합은 2이고, 이진수 10에 해당하는 Cout=1, s=0이 나타났다. 70~80ns에서도 세 입력의 합이 3이므로 Cout=1, s=1이 나타났다. 이는 예상 진리표와 일치하는 값이다.

6. 실험 3 · 4비트 가산기

핵심 내용

·설계 파일: adder_4bit.v

·테스트벤치: tb_adder_4bit.sv

·입력: a[3:0], b[3:0] / 출력: s[3:0], cout

·검증 조건: 256가지 입력 조합, 10ns 간격, 총 2560ns

핵심 코드

```

1 module adder_4bit(
2     input wire [3:0] a, input wire [3:0] b,
3     output wire [3:0] s, output wire cout
4 );
5     assign {cout, s} = a + b;

```

그림 9: 4비트 가산기 Verilog 핵심 코드

예상 결과

a	b	전체합	Cout	s
0	0	0	0	0
1	F(15)	16	1	0
F(15)	F(15)	30	1	E(14)

a, b의 덧셈의 결과에서 최상위 비트는 Cout, 하위 4비트는 s에 저장되므로 15를 16진수로 표현한 F 간의 덧셈은 위의 결과로 예상된다.

결과 설명

```

LAB1_PASS adder_4bit cases=256
D:\Engineering\Verilog_Lab\LAB1\lab1_03_adder4\sim\tb_adder_4bit.sv:54: $finish called at 2560000 (1ps)

```

그림 10: 4비트 가산기 시뮬레이션 실행 결과(PASS)

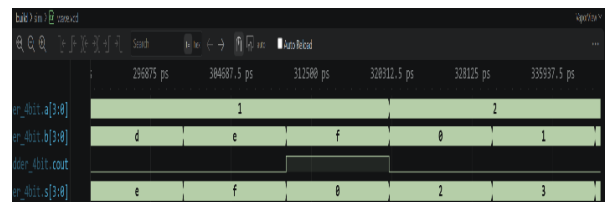


그림 11: 4비트 가산기의 합과 자리올림 발생 구간 파형

310~320ns에서 a=1, b=F가 입력되어 전체 합은 16이 되었다. 따라서 하위 4비트 출력 s는 0이되고 최상위 자리올림 Cout은 1이 되었다. 320ns 이후에는 a=2, b=0으로 바뀌면서 s=2, Cout=0으로 바뀌었다. 256가지 입력 조합이 모두 자동 검사(PASS)를 통과하였고, 예상 결과와 같다.

7. 실험 4 · 4비트 감산기

핵심 내용

·설계 파일: sub_4bit.v

·테스트벤치: tb_sub_4bit.sv

·입력: a[3:0], b[3:0] / 출력: d[3:0], bor

·검증 조건: 256가지 입력 조합, 10ns 간격, 총 2560ns

핵심 코드

```

8     assign d = a - b;
9     assign bor = (a < b);

```

그림 12: 4비트 감산기 Verilog 핵심 코드

예상 결과

a	b	계산	d	bor
0	0	0-0	0	0
0	1	0-1	F	1
F(15)	0	15-0	F	0

a-b의 연산 결과를 구할 때 d에는 뺄셈 결과의 하위 4비트가 저장되고, a가 b보다 작아 빌림이 필요하면 bor=1이 된다. 그 예로 0-1= -1이므로 bor=1을 빌려오고 4비트 차는 F(15)로 나타난다.

결과 설명

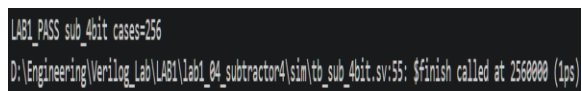


그림 13: 4비트 감산기 시뮬레이션 실행 결과(PASS)

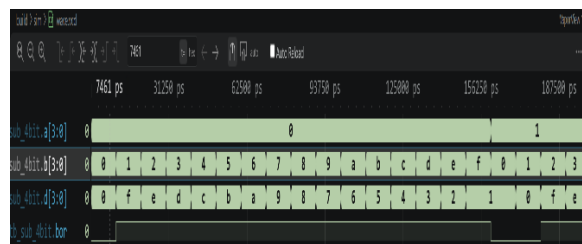


그림 14: 4비트 감산기의 차와 빌림 출력 파형

10~20ns에서 a=0, b=1이므로 빌림이 필요하여 bor=1이 되었고, 4비트 차는 F로 나타났다. 이는 예상 진리표의 결과와 동일했다. 이와 반대로 170~180ns에서 a=1, b=1이므로 차는 0이고 빌림이 필요하지 않아 bor=0이 되었다. 따라서 a<b 일때만 bor의 값이 1이 됨을 알 수 있다.

8. 실험 5 · 4비트 비교기

핵심 내용

·설계 파일: compare_4.v

·테스트벤치: tb_compare_4.sv

·입력: a[3:0], b[3:0] / 출력: o[2:0]

·검증 조건: 256가지 입력 조합, 10ns 간격, 총 2560ns

핵심 코드

```

7      assign o[2] = (a > b);
8      assign o[1] = (a == b);
9      assign o[0] = (a < b);

```

그림 15: 4비트 비교기 Verilog 핵심 코드

예상 결과

a	b	관계	o
0	0	a=b	010
0	1	a<b	001
1	0	a>b	100

a, b간의 크기 비교를 통해서 a가 b보다 클 때 o[2] = 1이 되므로 파형에서는 4로 나타나고, 같을 때 2, b가 더 크면 1로 출력이 나타날 것으로 예상된다.

결과 설명



그림 16: 4비트 비교기 시뮬레이션 실행 결과(PASS)

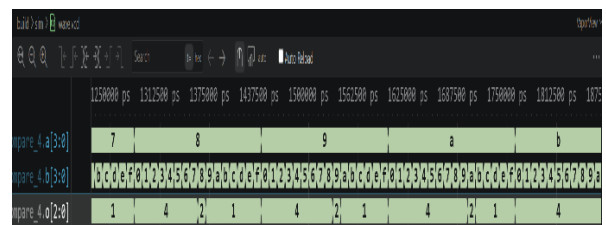


그림 17: 두 입력의 크기에 따른 비교기 출력 파형

파형에서 a=8일 때 b=7, 8, 9가 차례로 인가되었다. 이때 출력이 4, 2, 1로 나타났고 이는 2진수로 100, 010, 001과 같고 예상 진리표의 값과 일치하였다.

9. 실험 6 · 4:1 멀티플렉서

핵심 내용

·설계 파일: mux_4x1.v

·테스트벤치: tb_mux_4x1.sv

·입력: i[3:0], s[1:0] / 출력: z

·검증 조건: 64가지 입력 조합, 10ns 간격, 총 640ns

핵심 코드

```

7      assign z = (s == 2'b00) ? i[3] :
8                (s == 2'b01) ? i[2] :
9                (s == 2'b10) ? i[1] :
10               i[0];

```

그림 18: 4:1 멀티플렉서 Verilog 핵심 코드

예상 결과

i	s	선택되는 입력	Z
1000	00	i[3]	1
1000	01	i[2]	0

0001 11 i[0] 1
z값의 결정은 s값에 따라 결정된다. S가 00, 01, 10, 11이면 각각 i 값의 비트위치에 맞는 값이 z 값으로 들어간다.

결과 설명

LAB1_PASS mux_4x1 cases=64
D:\Engineering\Verilog_Lab\LAB1\lab1_06_mux4to1\sim\tb_mux_4x1.sv:52: \$finish called at 640000 (1ps)

그림 19: 4:1 멀티플렉서 시뮬레이션 실행 결과(PASS)

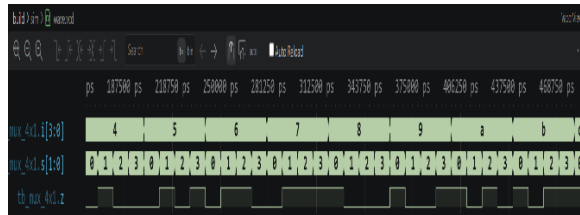


그림 20: 선택 신호에 따른 멀티플렉서 출력 파형

320~360ns에서 입력 i=1000(8)이 유지된 상태로 선택 신호가 00-01-10-11 순서대로 변경되었다. Z 값은 각각 1, 0, 0, 0 순서대로 출력되고, 이는 예상결과와 일치한다.

10. 실험 7 · 1:8 디멀티플렉서

핵심 내용

·설계 파일: demux_1x8.v

·테스트벤치: tb_demux_1x8.sv

·입력: i, s[2:0] / ·출력: o[7:0]

·검증 조건: 16가지 입력 조합, 10ns 간격, 총 160ns

핵심 코드

```
7 assign o = i
8      ? (8'b10000000 >> s) : 8'b00000000;
```

그림 21: 1:8 디멀티플렉서 Verilog 핵심 코드

예상 결과

i	s	o[7:0]
0	모든 값	00000000
1	000	10000000
1	111	00000001

Verilog 코드를 확인하면 입력 i의 값이 0이면 출력이 0, 1이면 s의 값에 따라 shift 이동을 통해 출력 값이 정해진다. 따라서 예상 진리표와 같은 결과를 예상할 수 있다.

결과 설명

LAB1_PASS demux_1x8 cases=16
D:\Engineering\Verilog_Lab\LAB1\lab1_07_demux1to8\sim\tb_demux_1x8.sv:54: \$finish called at 160000 (1ps)

그림 22: 1:8 디멀티플렉서 시뮬레이션 실행 결과(PASS)



그림 23: 선택 신호에 따른 디멀티플렉서 출력 위치 변화

입력 i=0인 80ns까지는 출력이 모두 0이었다. 입력 i=1인 상황에서는 s 값이 0에서 7까지 증가함에 따라 출력이 80-40-20-10-08-04-02-01로 이동하고, 이는 shift의 결과로 예상한 결과와 일치하였다.

11. 실험 8 · 8:3 인코더

핵심 내용

·설계 파일: encoder8x3.v

·테스트벤치: tb_encoder8x3.sv

·입력: i[7:0] / ·출력: a[2:0]

·검증 조건: 256가지 입력 조합, 10ns 간격, 총 2560ns

핵심 코드

```
case (i)
8'b10000000: a = 3'b000; 8'b01000000: a = 3'b001;
8'b00100000: a = 3'b010; 8'b00010000: a = 3'b011;
8'b00001000: a = 3'b100; 8'b00000100: a = 3'b101;
8'b00000010: a = 3'b110; 8'b00000001: a = 3'b111;
default: a = 3'b000;
endcase
```

그림 24: 8:3 인코더 Verilog 핵심 코드

예상 결과

i[7:0]	a[2:0]	의미
10000000	000	왼쪽 입력, 번호 0
00000001	111	오른쪽 입력, 번호 7
00000011	000	무효 입력

코드를 분석하면 입력 값 i는 1이 나타나는 왼쪽 비트부터 0~7까지의 출력 값을 나타낸다. 1이 여러 번 나타나는 경우는 무효 입력으로 default를 주므로 000의 출력이 예상된다.

결과 설명

LAB1_PASS encoder8x3 cases=256
D:\Engineering\Verilog_Lab\LAB1\lab1_08_encoder8to3\sim\tb_encoder8x3.sv:64: \$finish called at 2560000 (1ps)

그림 25: 8:3 인코더 시뮬레이션 실행 결과(PASS)

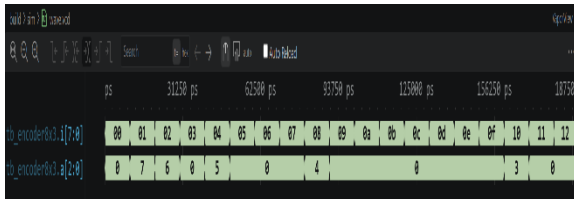


그림 26: 유효한 하나의 입력과 그에 따른 인코더 출력

입력 i=01일 때 출력 a=7, i=02일 때 출력 a=6인 것을 보면 입력 값에 1이 한번 포함된 경우 예상한 결과와 동일하게 나타났고, i=05와 같이 0000101인 경우 1이 두 번 나타나므로 default인 0이 출력되고, 이 또한 예상과 같다.

12. 실험 9 · 3:8 디코더

핵심 내용

·설계 파일: decoder3x8.v

·테스트벤치: tb_decoder3x8.sv

·입력: a, b, c / 출력: o[7:0]

·검증 조건: 8가지 입력 조합, 10ns 간격, 총 80ns

핵심 코드

```
8 assign o = 8'b00000001 << {a, b, c};
```

그림 27: 3:8 디코더 Verilog 핵심 코드

예상 결과

abc	O[7:0]
000	00000001
011	00001000
111	10000000

코드를 보면 a,b,c에 입력되는 값에 따라 00000001의 값을 왼쪽으로 비트 shift 시켜 출력 값을 결정한다. 따라서 111인 경우 7칸 이동하여 10000000으로 값이 예상된다.

결과 설명

LAB1_PASS decoder3x8 cases=8
D:\Engineering\Verilog_Lab\LAB1\lab1_09_decoder3to8\sim\tb_decoder3x8.sv:51: \$finish called at 80000 (1ps)

그림 28: 3:8 디코더 시뮬레이션 실행 결과(PASS)

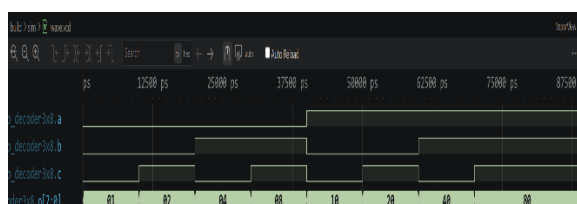


그림 29: 3 비트 입력에 따른 출력 파형

입력 abc 값이 증가함에 따라 출력 값이 01-02-04-08-10 순서대로 증가하고, 16진수인 결과 값을 2진수로 바꾸면 예상 진리표에서 구한 값과 동일하게 1의 값이 왼쪽으로 shift되는 결과임을 알 수 있다.

13. 실험 10 · 7세그먼트 디코더

핵심 내용

·설계 파일: seg_decoder.v

·테스트벤치: tb_seg_decoder.sv

·입력: bcd[3:0] / 출력: seg_data[7:0]

·검증 조건: 16가지 입력 조합, 10ns 간격, 총 160ns

핵심 코드(부분만 포함)

```
7 case (bcd)
8     4'h0: seg_data = 8'hFC;
9     4'h1: seg_data = 8'h60;
10    4'h2: seg_data = 8'hDA;
11    4'h3: seg_data = 8'hF2;
```

그림 30: 7세그먼트 디코더 Verilog 핵심 코드

예상 결과

bcd	Seg_data	표시
0	FC	0
1	60	1
8	FE	8

비트 순서가 {a, b, c, d, e, f, g, dp} 이므로, bcd가 0일 때 g 값만 빼고 전체 값이 1로 켜진다. 따라서 11111100이므로 16진수로는 FC로 Seg_data 값이 나타날 것이다. Dp는 0으로 값이 고정인 것을 코드를 통해 알 수 있다. 나머지 값 또한 동일한 방법으로 얻을 수 있다.

결과 설명

LAB1_PASS seg_decoder cases=16
D:\Engineering\Verilog_Lab\LAB1\lab1_10_seven_segment\sim\tb_seg_decoder.sv:63: \$finish called at 160000 (1ps)

그림 31: 7세그먼트 디코더 시뮬레이션 실행 결과(PASS)



그림 32: 0~F 입력에 따른 7세그먼트 점등 패턴

입력이 8일 때의 출력 값을 확인해보자. 입력이 8이면 전체 값이 모두 1로 커지므로 11111110이고 16진수로 FE이고, 결과 값이 동일하므로 예상한 동작원리와 일치한다.

14. 코드 수정 및 실패 · 복구 검증

실패 · 복구 검증

각 실험에서 매뉴얼에 제시된 RTL 코드의 일부를 의도적으로 변경하고, 테스트 벤치의 기대 값은 수정하지 않은 상태로 시뮬레이션을 진행하였다. 변경 후에 예상 입력에 대한 FAIL 여부를 확인했고, 이후 RTL을 원래 코드로 복구하여 다시 PASS가 되었음을 확인하였다. 실패와 복구 단계의 로그 및 파형 파일은 GitHub 저장소에 기록하였다.

	변경 내용	실패 확인
1	XOR→OR	입력11: 기대 110, 실제 111
2	Cout: OR→AND	a=0, b=1, cin=1: 기대 10, 실제 00
3	a+b→a-b	a=0, b=1: 기대 cout=0, s=1, 실제 cout=1, s=F
4	Bor 조건 < → <=	a=0, b=0: 기대 bor = 0, 실제 bor=1
5	== → !=	a=0, b=0: 기대 o = 010, 실제 o = 000
6	선택 입력 반전	i=0001, s=00: 기대 z=0, 실제 z=1
7	>> → <<	i=1, s=1: 기대 01000000, 실제 00000000
8	00000001의 출력 111 → 110	기대 a=111, 실제 a= 110
9	<< → >>	abc=001: 기대 00000010, 실제 00000000
10	8'hFC → 8'h60	Bcd=0: 기대 FC, 실제 60

실패 · 복구 기록

실험	실패 · 복구 기록
1	실패: a07dc21..., 실패로그 복구: a751228..., 복구로그
2	실패: 19e9237..., 실패로그 복구: 3f184dc..., 복구로그
3	실패: 1447b9f..., 실패로그 복구: ce358df..., 복구로그
4	실패: 45da1b3..., 실패로그 복구: 8ca84bb..., 복구로그
5	실패: 5512feb..., 실패로그 복구: 4b2b3f3..., 복구로그
6	실패: f9155fc..., 실패로그

	복구: c1ed886..., 복구로그
7	실패: 6cc7ff0..., 실패로그 복구: e686acc..., 복구로그
8	실패: c7aa124..., 실패로그 복구: 48a26a2..., 복구로그
9	실패: dd4254e..., 실패로그 복구: 72421d0..., 복구로그
10	실패: c203c39..., 실패로그 복구: 372a510..., 복구로그

15. 보드 검증 계획

실제 실험에서는 Vivado에서 합성 및 구현을 수행한 뒤 FPGA 보드를 프로그래밍 한다. 이후 스위치와 버튼으로 입력을 변경하여 LED 및 7세그먼트 출력을 확인하고 시뮬레이션 결과와 비교한다. 또한 보드 검증 결과는 입력과 출력 장치가 함께 보이는 사진 및 동작 영상으로 기록하고, 예상결과와의 일치 여부를 실험 후 보고서에 정리한다.

16. 결론

Lab1에서는 Verilog HDL로 10개의 조합논리회로를 설계하고 테스트 벤치를 이용해 입력 조합에 따른 출력을 검증하였다. 각 테스트 벤치는 예상 값과 실제 값을 자동으로 비교하여 모든 실험에서 지정된 검사 횟수와 PASS 결과를 확인하였다. 또한, VaporView 파형을 통해 논리 연산, 산술 연산, 선택 신호, 부호화 및 7세그먼트 출력의 변화를 직접 확인하였고 예상한 시뮬레이션 결과와 일치함을 확인하였다. 이후 실험에서는 동일한 RTL을 Vivado에서 구현하고 실제 FPGA 보드에서 입출력과 비교하였다.

참고 문헌

[1] I. Kuon and J. Rose, "Measuring the Gap Between FPGAs and ASICs," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 26, no. 2, pp. 203–215, Feb. 2007, doi: 10.1109/TCAD.2006.884574.

[2] S. M. Trimberger, "Three Ages of FPGAs: A Retrospective on the First Thirty Years of FPGA Technology," Proceedings of the IEEE, vol. 103, no. 3, pp. 318–331, Mar. 2015, doi: 10.1109/JPROC.2015.2392104.